SIMATS ENGINEERING

ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyse virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2)

Question Count: 5 **Total Marks: 25**

Code of Conduct

I B.V.THARUN (192525155) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Assessment Tasks Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.
2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.
4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

A) Use case: During seasonal sales such as a festival offer or a year-end sale, an online store may receive many times its normal traffic within a few minutes, and the exact size and timing of the surge are very hard to predict. If the company runs a fixed number of servers, it faces a poor choice: either the servers are sized for the average day and the website slows down or crashes at the peak, which means downtime, frustrated customers and lost sales, or they are sized for the peak and remain idle and wasteful for the rest of the year. Cloud **auto-scaling** and **load balancing** solve both problems together, because capacity is added automatically when demand rises, traffic is shared evenly among the available servers, and the extra capacity is removed when the sale ends.

1. How auto-scaling and load balancing handle the demand:

The solution is built from four cooperating components, and each of them removes a different kind of bottleneck. The load balancer is the single entry point for customers. It receives every request and distributes it across many web and application servers located in several availability zones, using methods such as round-robin or least-connections. It also performs health checks, so a server that fails or becomes too slow is taken out of rotation automatically and users are never sent to it, and it can route requests by URL, for example sending the checkout path to a dedicated and better protected group of servers. The auto-scaling group works behind it and changes the number of servers according to demand, as summarized in the table below.

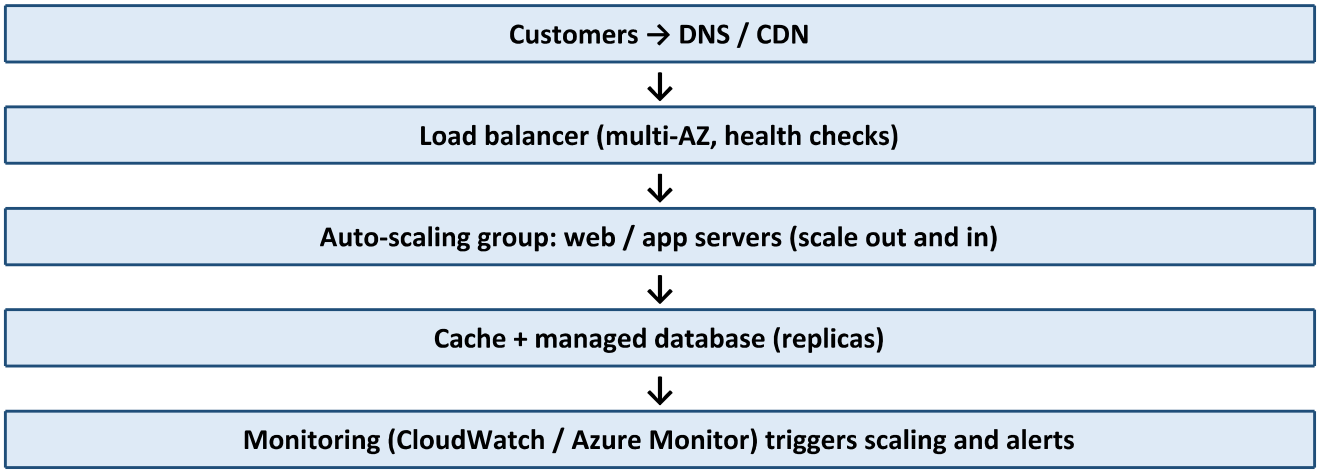
Table: How auto-scaling and load balancing handle the demand

Component	Working
Load balancer	Receives all user requests and spreads them across many web/application servers in several availability zones (round-robin or least-connections). Health checks remove a failed server automatically, so users are never sent to it. It also offloads TLS and can route by URL (e.g. /checkout to a dedicated pool).
Auto-scaling group	Launches extra servers (scale-out) when metrics such as CPU, requests per second or response time cross a threshold, and terminates them (scale-in) when traffic falls. Rules: **dynamic/target tracking** for sudden spikes, **scheduled scaling** for known sale start times, **predictive scaling** from past patterns. Minimum and maximum limits control cost.
Stateless design and caching	Sessions and carts are kept in a shared cache/database (e.g. Redis) and static images in a CDN, so any new server can serve any user immediately.
Database layer	Read replicas, managed database with auto-scaling storage and connection pooling prevent the database from becoming the bottleneck.

The table shows that the strength of the design lies in the combination of the components. Auto-scaling can be configured in three complementary ways. Dynamic or target-tracking scaling reacts to live metrics such as CPU utilization, requests per second or response time and adds servers within minutes when a sudden spike occurs. Scheduled scaling prepares capacity before a known event, for example a few minutes before the sale starts at midnight, so that the servers are already running when the first customers arrive. Predictive scaling studies the traffic of earlier sales and forecasts the demand, so capacity can be increased even before the metrics rise. Minimum and maximum limits keep the fleet large enough to be safe and small enough to control the cost.

For the scaling to work smoothly, the application must be designed as stateless. Shopping carts and user sessions are stored in a shared in-memory cache such as Redis or in a database, and not on a single web server, so that any newly launched server can serve any customer immediately. Static content such as product images, style sheets and scripts is delivered through a CDN, which removes a large part of the load from the servers. The database tier is protected in the same way, using read replicas for product browsing, a managed database with automatically growing storage, and connection pooling, so that the database does not become the bottleneck when the web tier has scaled out. The following flow chart shows how a customer request travels through these layers.

Flowchart: Auto-scaling and load balancing for a seasonal sale



2. Reduced downtime and better user experience:

This approach reduces downtime because it removes single points of failure. Requests are handled by multiple servers in multiple availability zones, so when one server or even one zone fails, the load balancer bypasses it and the customers do not notice any interruption. The system is also self-healing, since the auto-scaling group continuously compares the number of healthy servers with the desired number and automatically replaces any instance that has failed its health check. In a fixed-capacity design such a failure would need a manual repair during the busiest hours of the year.

The user experience improves because performance stays consistent. As the number of visitors grows, the capacity grows with it, so product pages continue to load quickly, the search results remain responsive and the checkout does not time out. This has a direct business effect, because slow pages and failed payments are the main reasons why customers abandon their carts. By keeping the site available and fast during the peak, the company protects its sales and its reputation. At the same time, the approach is cost-efficient, because the company pays for the additional servers only for the hours in which the sale actually runs and releases them afterwards.

3. Role of monitoring tools:

Monitoring tools such as Amazon CloudWatch, Azure Monitor, Google Cloud Monitoring and Prometheus with Grafana provide the information on which the whole setup depends. They continuously collect metrics such as CPU use, memory, response latency, error rate, request count and queue length, together with application and server logs. This data drives the auto-scaling policies, because the scaling rules are triggered by the metrics, and it raises alarms to the operations team when thresholds are crossed, for example when payment errors increase or when database connections are nearly exhausted.

During the sale, dashboards give the team a live view of traffic, orders and system health, which helps them find bottlenecks such as slow database queries or a saturated cache and fix them before customers

are affected. After the event, the collected data is used for capacity planning and for tuning the scaling thresholds, so the next sale is handled even better. Without monitoring, auto-scaling would have nothing to react to and the team would not know about problems until customers complained.

4. Real-world e-commerce scenario:

Consider an online retailer such as Amazon or Flipkart that starts a festival sale at midnight. Before the sale, scheduled scaling adds servers so that the platform is ready. When millions of users arrive, the load balancer spreads the traffic across the servers in several zones and target-tracking scaling adds more web servers within minutes, while the CDN serves product images and the cache serves product and session data quickly. Alarms warn the team if payment errors or response times rise, so they can react immediately. Once the sale ends and traffic falls, scale-in removes the extra servers, so the company pays only for what it used and keeps its infrastructure cost under control.

5. Best practices and practical considerations:

To obtain the full benefit of auto-scaling during a sale, the company should test its set-up before the event. Load-testing tools simulate thousands of simultaneous shoppers so that the team can verify that scaling policies react quickly enough, that the database and payment gateway can handle the peak, and that no hidden limit, such as a cap on the number of instances or on network addresses, stops the scale-out. Cool-down periods and sensible thresholds prevent the fleet from growing and shrinking too often, and warm-up scripts or pre-built machine images make a new server ready to serve customers within seconds.

It is also wise to prepare graceful degradation. If the load exceeds even the expected peak, non-essential features such as recommendations or reviews can be switched off temporarily, and a queue can hold orders for a short time, so the critical path of browsing, cart and payment continues to work. Together with budget alerts that warn about unusually high spending, these practices make the elastic set-up both reliable and financially predictable.

Conclusion: Load balancing and auto-scaling, guided by monitoring, give elastic capacity, high availability and a smooth shopping experience at an optimized cost, which is exactly what a business with unpredictable seasonal traffic needs.

2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.

A) Use case: The organization must keep large volumes of backup data, such as databases, file servers and virtual machine images, for a long period of time. The backups must never be lost, must be protected from unauthorized access and from ransomware, and should cost less than maintaining a tape library or a storage area network (SAN) in its own data center. Cloud storage services such as Amazon S3, Azure Blob Storage and Google Cloud Storage are designed for exactly this purpose.

1. Durability and data redundancy in cloud storage:

Durability means that stored data is not lost or corrupted over time, and cloud providers achieve it through redundancy. Object storage automatically stores each object as multiple copies on several devices and, in the standard storage classes, across multiple availability zones, so that the failure of a disk, a server or even a whole data center does not cause data loss. Providers publish very high durability targets; for example, Amazon S3 is designed for 99.999999999% durability, which is commonly called "eleven nines". Checksums are used to detect silent corruption, and when a damaged or missing copy is found, the system rebuilds it automatically from the healthy copies, so the data repairs itself without human involvement.

Customers can also choose the level of redundancy according to the importance of the data and the budget. Locally redundant storage keeps copies in one data center, zone-redundant storage spreads them across zones in a region, and geo-redundant storage replicates them to a second region hundreds of kilometers away, which protects against a regional disaster. In addition, versioning keeps older versions of every object, and object lock makes backups immutable using the WORM (write once, read many) model, so they cannot be changed or deleted accidentally or by an attacker during the retention period. Lifecycle management rules then move aging backups automatically to cooler and cheaper tiers, so durability is retained while the cost falls.

2. Protecting backup data with encryption and access control:

Backups often contain the most sensitive information of an organization, so they need several layers of protection, which are summarized in the table below.

Table: Protecting backup data with encryption and access control

Control	How it protects backups
Encryption in transit	TLS/HTTPS (or VPN) while uploading and restoring so data cannot be intercepted.
Encryption at rest	AES-256 server-side encryption, or client-side encryption before upload; keys managed in a Key Management Service/HSM (customer-managed keys, rotation, separate key permissions).
Identity and access management	Least-privilege IAM roles/policies, multi-factor authentication, separate backup account, no public access (block public buckets), bucket policies, MFA-delete and short-lived credentials.
Monitoring and audit	Access logs and audit trails (CloudTrail), alerts on unusual deletes or downloads.
Network protection	Private endpoints / VPC endpoints and IP restrictions.

Encryption protects the confidentiality of the data. In transit, TLS or HTTPS, or a VPN, prevents interception while backups are uploaded and restored. At rest, AES-256 encryption is applied, either by the storage service on the server side or by the customer on the client side before upload, and the encryption keys are managed in a Key Management Service or hardware security module with rotation and separate permissions. With customer-managed keys, the organization stays in control, and deleting or disabling a key makes the corresponding data unreadable.

Access control decides who can reach the data at all. Least-privilege IAM roles and policies give each person and application only the permissions they need, multi-factor authentication protects administrators, and public access to the storage buckets is blocked. Placing backups in a separate cloud account, enabling MFA-delete and using short-lived credentials limit the damage if one account is compromised, and private endpoints with IP restrictions keep the traffic off the public internet. Finally, access logs and audit trails such as CloudTrail record every read and delete, and alerts warn the security team about unusual activity, for example a sudden mass deletion or a large download.

3. Cost advantages over traditional storage:

The cloud changes the economics of backup storage in several ways, as the comparison below shows.

Table: Cost comparison of traditional and cloud storage

Aspect	Traditional storage (tape / SAN / own data center)	Cloud storage

Capital cost	High upfront purchase of hardware	No upfront cost; pay-as-you-go (OPEX)
Capacity planning	Over-provisioning for future growth	Pay only for the capacity used; scales without limit
Maintenance	Staff, power, cooling, space, hardware replacement	Handled by the provider
Redundancy / DR site	Needs a second site and extra hardware	Built-in replication, little extra cost
Tiering	Manual	Automatic lifecycle tiers (hot, cool, archive) with very low archive prices

The most important difference is that traditional storage requires a large capital investment before a single byte is stored, and the capacity must be planned years ahead, so organizations normally over-buy and leave part of it unused. Cloud storage turns this into an operating expense. The organization pays only for the capacity it actually uses, the capacity can grow without any limit, and no hardware needs to be purchased, installed or replaced. Power, cooling, floor space and the administrators needed to maintain the equipment are the responsibility of the provider and are shared among millions of customers, which reduces the cost per gigabyte.

Redundancy and disaster recovery also become cheaper. In a traditional setup a second site with duplicate hardware is needed to protect against a disaster, while in the cloud the replication to another region is a configuration option and a small extra charge. Automatic lifecycle tiering moves old backups to archive tiers whose price is very low, and the organization can retrieve them when needed. Taken together, these factors normally give a much lower total cost of ownership for long-term backup data.

4. Disaster recovery use case:

Consider a hospital group that backs up its patient-record database and imaging files every night to encrypted, versioned, cross-region cloud storage with object lock enabled. One day a ransomware attack encrypts all the servers in its own data center. Because the cloud backups are immutable and located in another region, the attackers cannot touch them. The IT team restores the last clean backup into new cloud servers, or into rebuilt on-premises servers, within the agreed recovery time objective, and loses only the data created after the last backup, which is the recovery point objective. Without the cloud copy, the hospital would have lost its records permanently or would have been forced to pay the ransom.

5. Best practices for managing cloud backups:

A good backup strategy follows the 3-2-1 rule: keep at least three copies of the data, on two different types of storage, with one copy in a different location. Cloud storage makes this rule easy to satisfy, because a copy in another region, or even in another provider, can be created through simple policies. The backup process should be automated and scheduled, and its success must be monitored, because a backup job that fails silently is as dangerous as having no backup at all.

Just as important is regular restore testing. The organization should restore sample data at fixed intervals to confirm that the backups are complete, that the encryption keys are available and that the recovery time meets the target. Retention rules should follow legal and business requirements, so data is not kept longer than necessary or deleted too early. By combining automation, the 3-2-1 rule and regular tests, the organization can be confident that its cloud backups will really work when a disaster happens.

Conclusion: Cloud storage gives durable, redundant, encrypted and access-controlled backup at a lower cost and with faster recovery than traditional systems, which makes it the preferred choice for protecting large volumes of sensitive backup data.

3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

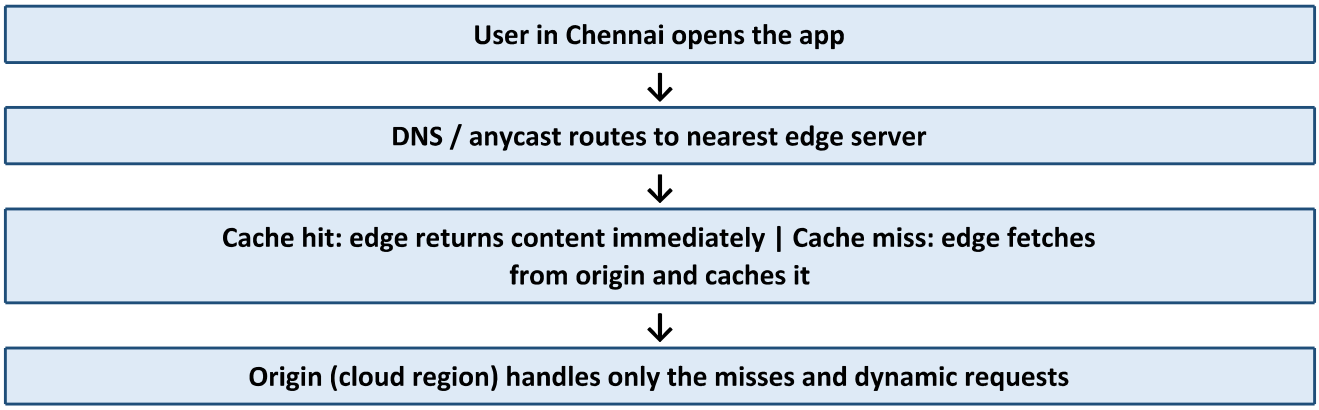
A) Use case: The users of the mobile app are spread across several continents. If everything is served from a single data center, distant users suffer high **latency**, which is caused by the physical distance and the number of network hops between the user and the server. They also experience slow loading and buffering, while the origin server becomes overloaded by requests from all over the world. A Content Delivery Network (CDN) with edge servers, supported by the global footprint of a cloud provider, solves these problems by bringing the content and part of the processing close to every user.

1. How CDNs improve performance and reduce latency:

A CDN is a geographically distributed network of servers, called points of presence or edge locations. Copies of static content such as images, video segments, JavaScript files, app updates and even cacheable API responses are stored on these servers, and when a user makes a request it is answered by the nearest edge server instead of the distant origin. Because the data travels a much shorter distance and passes through fewer network hops, the round-trip time falls significantly, and the app starts and loads faster.

A CDN also reduces the load on the origin and the bandwidth cost, because most requests are answered from the cache and only cache misses and dynamic requests reach the origin. In addition, it optimizes delivery through compression, modern protocols such as HTTP/2 and HTTP/3, TLS termination at the edge, persistent connections and optimized routing across the provider's network. Spreading traffic over many servers also improves availability and security, because edge locations can absorb DDoS attacks and apply a web application firewall before the traffic reaches the application. The request path is shown in the following flow chart.

Flowchart: CDN request flow



2. How edge servers enhance user experience across regions:

Edge servers make the experience of a user in Chennai, Frankfurt or Sao Paulo similar, even though the origin is in only one or two regions. App start-up is faster, pages and assets load quickly, and media playback is smoother with less buffering. When a country experiences a local traffic peak, the load is absorbed by the local edge servers and does not travel to the origin or affect users elsewhere.

Modern edge platforms such as CloudFront Functions, Lambda@Edge and Cloudflare Workers can also run small pieces of code near the user. This edge computing is used for authentication checks, redirects, personalization and A/B testing, so these tasks are performed with minimal delay. The edge network is also

resilient: if a local network link is congested or an edge node fails, requests are automatically redirected to other nearby edge servers, and the user continues without any visible interruption.

3. How cloud providers support global availability:

Cloud providers support global availability through a combination of infrastructure and services. They operate regions and availability zones around the world, which allow the company to deploy the application close to each group of users and to satisfy local data-residency laws. Global load balancing and DNS services, using latency-based or geo routing and anycast addresses, send every user to the nearest healthy region, and if a region fails the traffic is redirected to another one.

Data is kept close to users and protected through multi-region replication, such as global databases and cross-region storage replication. Providers also operate private global backbone networks that connect their regions and edge locations, giving faster and more reliable paths than the public internet. Within and across regions, auto-scaling and failover mechanisms keep the service running during failures and traffic peaks. Together, these capabilities allow even a small mobile app company to offer a worldwide service without building its own global infrastructure.

4. Real-time streaming / gaming example:

For a streaming application, such as a video platform that uses its own CDN, or any service that uses CloudFront or Akamai, the video segments are stored on edge servers near the viewers. Playback therefore starts quickly and rebuffering is low even when a very popular title is released. For a multiplayer mobile game, the large patches, maps and assets are downloaded from the CDN, while the live game sessions run on game servers in the nearest regional cluster selected by global routing, which keeps the ping low and the gameplay smooth. It is important to note that real-time game traffic itself cannot be cached, so low latency in the match comes from regional game servers, while the CDN speeds up the assets and updates around it.

5. Design considerations for a global mobile application:

Besides the CDN and the global regions, the mobile app company must design the application itself for worldwide use. Static assets should be versioned and given suitable cache lifetimes so that the edge servers keep them for as long as possible, while dynamic data is kept small and compressed so that it travels quickly over mobile networks, which are often slow or unstable. The app can also use techniques such as offline caching, retry with back-off and adaptive quality, so that it continues to work in places with weak connectivity.

Operationally, the company should measure the real experience of users in each region through real-user monitoring and synthetic tests from different countries, and use the results to decide where new edge locations or regional deployments are needed. Compliance with local laws, such as data-residency rules and consent requirements, must be considered when choosing the regions. With this combination of technical and operational planning, the application can grow into new markets with predictable performance and little additional effort.

Conclusion: CDNs and edge servers deliver content from nearby locations, and the provider's global regions, routing and replication give the application worldwide performance and availability, which results in a fast and consistent experience for every user.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.

A) Use case: A bank or other financial institution handles sensitive transactions and customer data, and it is subject to strict rules, such as PCI-DSS for card data, ISO/IEC 27001, SOC 2, GDPR for European data and national regulator requirements like the RBI directives in India. It must be able to prove that it controls its systems, keep a complete audit trail of activity and ensure that data stays only in the countries where the law permits. The cloud can meet these needs, but only if the provider's capabilities and the customer's own controls are combined correctly.

1. How cloud providers support regulatory compliance and auditing:

Major cloud providers undergo independent audits and publish certifications and reports such as ISO 27001, SOC 1, 2 and 3, PCI-DSS and HIPAA. Customers can download these reports from portals such as AWS Artifact, Azure Service Trust Portal and Compliance Manager, and Google Compliance Reports Manager, and use them as evidence during their own audits. This means that the financial institution does not need to audit the provider's physical data centers itself.

Compliance is organized through the shared responsibility model. The provider is responsible for securing the physical data centers, the hardware and the basic services, while the customer remains responsible for its data, identities, configuration and applications. Providers also supply tools that check this customer-side configuration continuously. Services such as AWS Config, Azure Policy and Google Security Command Center compare resources with standards, report violations and can even correct them automatically. Immutable activity logs, access reviews and compliance reports create an audit-ready record that regulators and auditors can verify easily.

2. Role of logging, monitoring and encryption in security:

Logging, monitoring, encryption and identity management are the four technical pillars of cloud security for a financial institution, and the table below summarizes the role of each.

Table: Role of logging, monitoring, encryption and IAM

Control	Role
Logging	Records who did what, when and from where (API calls, logins, data access, configuration changes), e.g. CloudTrail, Azure Activity Log. Logs are stored in tamper-proof storage and retained as required; they support audits and forensic investigation.
Monitoring	Continuous tracking of metrics and events; SIEM and threat detection (GuardDuty, Defender for Cloud) with real-time alerts for suspicious activity such as unusual logins, large transfers or fraud patterns.
Encryption	Protects confidentiality: TLS in transit, AES-256 at rest, field-level encryption/tokenization for card numbers and account data; keys in KMS/HSM with rotation and strict access, preferably customer-managed keys so the provider cannot read the data.
IAM	Least privilege, MFA, segregation of duties and privileged access management.

Logging answers the questions of who did what, when and from where. API calls, logins, data access and configuration changes are recorded by services such as CloudTrail and the Azure Activity Log, stored in tamper-proof storage and retained for the period required by regulation. These logs are the basis for audits and for forensic investigation after an incident. Monitoring uses the same data in real time. SIEM systems and threat-detection services such as GuardDuty and Microsoft Defender for Cloud analyze events and raise alerts for suspicious behavior, for example unusual logins, large data transfers or fraud patterns in transactions.

Encryption protects the confidentiality of data even if other controls fail. TLS protects data in transit and AES-256 protects data at rest, while field-level encryption or tokenization is used for especially sensitive items such as card numbers and account details. The keys are kept in a KMS or HSM with rotation and strict access control, and customer-managed keys are preferred, so the provider cannot read the data. Identity and access management completes the picture by applying least privilege, multi-factor authentication, segregation of duties and privileged access management, so that no single person can misuse the system without being noticed.

3. Ensuring data sovereignty:

Data sovereignty means that data remains subject to the laws of the country in which it is collected and is stored and processed only in approved locations. The first step is to choose cloud regions inside the permitted country, for example the Mumbai or Hyderabad regions for Indian data, and to store and process regulated data only there. This choice must then be enforced and not left to individual engineers, so organization-level policies such as Azure Policy, AWS Service Control Policies and organization policies in Google Cloud are used to deny the creation of resources, or the replication of data, outside the approved regions.

Backups and disaster recovery copies need the same care, because geo-redundancy must target only regions within the same jurisdiction. The institution can also own the encryption keys by using customer-managed keys or an external key manager located in the country, and it can revoke those keys to make the data unreadable if needed. Contractual and legal controls add another layer, including data-processing agreements, provider commitments on data location and access by foreign authorities, and the use of sovereign-cloud offerings where available. Finally, regular audits of data locations and access logs verify that these rules are actually followed.

4. Example involving sensitive financial transactions:

A bank runs its online payment service in a cloud region inside its own country. Card numbers are tokenized and encrypted with customer-managed keys, and every transaction call is written to an immutable log store. A monitoring system analyzes the transactions in real time and flags unusual patterns, such as many payments from a new location within a few minutes, and policies prevent any copy of the data from leaving the approved regions. When the regulator asks for evidence, the bank provides the cloud provider's PCI-DSS report together with its own audit logs and configuration compliance report. This protects the customers, keeps the data in the required jurisdiction and makes audits straightforward.

5. Governance and continuous compliance:

Regulatory compliance in the cloud is not a one-time activity, because regulations, threats and the institution's own systems keep changing. The institution should therefore establish a governance process in which responsibilities are clearly assigned, policies are written down, and every change to the environment is reviewed. Compliance rules are converted into code, so that unencrypted storage, open network ports or resources in non-approved regions are blocked automatically at the time they are created and not discovered months later in an audit.

Regular activities strengthen this process. Access rights are reviewed periodically, vulnerability scans and penetration tests are performed, incident response and disaster recovery plans are rehearsed, and staff receive training on secure practices. Evidence from all these activities is collected in a central place, so that when a regulator or an auditor asks for proof, the institution can supply it quickly. This continuous approach turns compliance from a periodic burden into a normal part of daily operations.

Conclusion: Certified providers, strong encryption and IAM, complete logging and monitoring, and region and key controls together allow a financial institution to use the cloud securely while meeting compliance and sovereignty rules.

5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

A) Use case: The development team wants faster and more consistent deployments and a better use of its servers. A container packages an application together with its libraries and dependencies into a single image that runs the same way everywhere, and an orchestration platform such as Kubernetes automates the running, scaling and recovery of many containers. Together they form the basis of modern cloud-native development.

1. Containers vs virtual machines:

A virtual machine includes a complete guest operating system running on top of a hypervisor, while a container shares the kernel of the host operating system and runs as an isolated process. This single architectural difference causes most of the differences in performance and resource usage, which are compared in the table below.

Table: Containers versus virtual machines

Parameter	Virtual machine	Container
Architecture	Each VM has its own full guest OS on top of a hypervisor	Containers share the host OS kernel and run as isolated processes (container engine)
Size	Gigabytes	Megabytes
Start-up time	Minutes (OS boot)	Seconds or less
Performance	Hypervisor and guest OS overhead	Near-native performance, little overhead
Resource usage	Each VM reserves CPU/RAM for its own OS, so fewer per host	Lightweight; many more containers per host, better utilization
Isolation	Strong, hardware-level isolation	Process-level isolation (namespaces, cgroups); weaker than VMs unless hardened
Best for	Different operating systems, strong isolation, legacy apps	Microservices, CI/CD, scalable cloud-native apps

In terms of performance, a VM suffers from the overhead of the hypervisor and of a full guest operating system, and it needs minutes to boot, whereas a container starts in seconds or even less and runs with near-native performance. In terms of resource usage, each VM reserves its own CPU and memory for its operating system and its image is measured in gigabytes, so only a limited number of VMs can run on one host. Container images are measured in megabytes and are very lightweight, so many more containers can be placed on the same host, which gives better utilization and lower cost.

The trade-off is isolation. A VM provides strong, hardware-level isolation and can run a different operating system from the host, which makes it better for legacy applications and for workloads of different tenants that require the highest level of separation. Containers provide process-level isolation using namespaces and control groups, which is weaker unless it is hardened with additional measures, but it is entirely

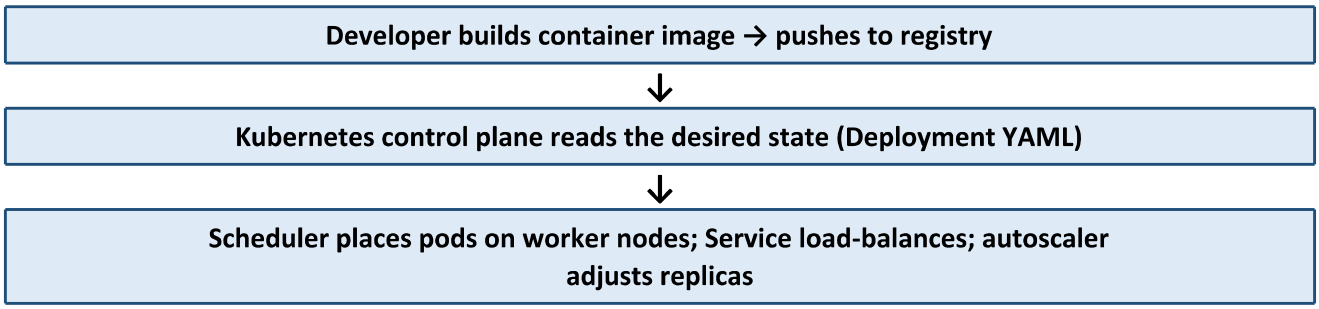
sufficient for the microservices and CI/CD pipelines of most cloud-native applications. In practice, many organizations run containers inside VMs and thus combine the strengths of both.

2. How orchestration (Kubernetes) simplifies deployment and scaling:

Running a few containers by hand is simple, but running hundreds of them across many servers is not. Kubernetes solves this by letting the team declare the desired state of the application, such as the image, the number of replicas and the resource limits, in YAML files. The control plane then schedules the containers onto suitable worker nodes and continuously works to keep the actual state equal to the declared state, which removes most of the manual work of deployment.

Scaling is automatic. The Horizontal Pod Autoscaler increases or decreases the number of containers according to the load, and the Cluster Autoscaler adds or removes worker nodes when more or less capacity is needed. Kubernetes is also self-healing: it restarts failed containers, reschedules them when a node fails and uses health probes to decide whether a container is ready to receive traffic. Built-in service discovery and load balancing give each group of containers a stable name and distribute traffic among them, while rolling updates, rollbacks, canary releases and blue-green deployments allow new versions to be released without downtime. Configuration, secrets, storage and resource limits are also managed in a consistent way. The deployment process is shown in the following flow chart.

Flowchart: Kubernetes deployment flow



3. Portability benefits across cloud environments:

The most important benefit of containerization for a multi-cloud strategy is portability, which is often summarized as "build once, run anywhere". The container image contains everything the application needs, so it behaves in the same way on a developer's laptop, in an on-premises data center and on AWS, Azure or Google Cloud. This also removes the familiar "it works on my machine" problem, because development, test and production environments become consistent.

Kubernetes strengthens this portability, since it is available as a managed service on all major clouds, namely EKS, AKS and GKE, and it exposes the same API everywhere. Applications and their manifests can therefore move from one provider to another with very little change. This reduces vendor lock-in, allows hybrid and multi-cloud deployments, and makes migration, disaster recovery and cost-based placement of workloads much easier.

4. Example of a microservices-based application:

An online shopping application can be split into independent services such as **User/Authentication, Product Catalogue, Cart, Order, Payment, Inventory and Notification**. Each service has its own container image, its own database and its own release cycle, and the services communicate through REST or gRPC calls or through a message queue, with an API gateway as the single entry point for users. Kubernetes runs several replicas of every service. During a sale, only the Catalogue and Cart services need to scale out, which saves resources, and if a new release of the Payment service has a fault, it can be rolled back without

touching any other service. This independence is what makes the microservices style, supported by containers and orchestration, so effective for large and fast-changing applications.

5. Challenges and best practices of containerization:

Containerization also brings challenges that the team must plan for. Container images must be built from trusted base images, scanned for vulnerabilities and kept small, and they should run as non-root users with limited privileges. Persistent data needs special attention, because containers are temporary by nature, so databases and other stateful services must use persistent volumes or managed cloud databases. The learning curve of Kubernetes is also considerable, and the team needs to invest in training, in monitoring tools and in clear conventions for configuration.

Good practices make the transition smoother. Applications should follow the twelve-factor principles, with configuration kept outside the image and logs written to standard output, and deployments should be automated through a CI/CD pipeline so that every release is tested, scanned and traceable. Resource requests and limits should be set for each container, and a service mesh or network policies can be added to control and secure the communication between microservices. With these practices, containers deliver their benefits of speed, efficiency and portability without losing security or stability.

Conclusion: Containers are lighter and faster than VMs, Kubernetes automates deployment, scaling and recovery, and container portability makes applications easy to run in any cloud, which together give the development team speed, consistency and flexibility.

Application Based Questions Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

Total Marks Awarded: